LMS Lite

從申請到結果：
一條可操作的實驗室流程

申請者 × Admin × 實驗人員

功能展示版 · Django / Heroku

報告者：101409492 - 華致齊

Demo：2026 / 10 / 17



專案測試網址

https://lms-lite-demo-6b4412ed2069.herokuapp.com/

註冊邀請碼

LMSDEMO

歡迎測試

專案簡介｜為什麼選擇 LMS (Laboratory Management System)？

實務上的問題

1. 申請資訊容易散落在不同人、不同紀錄中

2. 實驗工作進度難以追蹤、同步

3. 實驗排程、執行、結果發布需要不同角色協作，行政流程耗時費工且易影響時效性

專案的目標

1. 申請者可以開立申請、追蹤進度、查看結果

2. Admin 可以受理、退回、排程與發布結果

3. 實驗人員只處理被指派的排程與結果輸入

4. 每一次狀態轉換都可以被系統記錄

專案成果展示

1. 從註冊、送出申請一路走到結果發布

2. 三種身份看到不同工作台與可操作功能

3. 非法存取會被 403 / 404 阻擋

4. 展示資料可以快速 seed、reset，方便反覆演練

專案簡介｜功能目標

申請流程完整閉環

草稿 → 送出 → 受理 / 退回 → 排程 → 實驗 → 完成 → 發布，申請者在結果發布前看不到實驗結果

三種身份各自有清楚的邊界

Applicant (申請者)：只看自己的申請單及實驗結果
Lab Staff (實驗人員)：只看指派給自己的排程並回復結果
Admin (系統管理員)：負責受理/退回申請、排程、管理帳號與結果發布

後端規則保證流程正確

狀態由 service layer 控制，不讓 View 隨意修改，
每次狀態變更都留下 RequestStatusLog

展示資料可以重複使用

seed_demo 建立帳號、主檔與示範申請
reset_demo 將 Heroku 快速恢復到乾淨狀態

技術棧

Backend

Python 3.13
Django 5.2
Django Templates
Django auth / Groups / Permissions

Data Base

Local：SQLite
Cloud：Heroku Postgres

Web Server

Gunicorn

Static File Serving

WhiteNoise

Deployment

Heroku : Basic Dyno

- DATABASE_URL 切換資料庫
- Release phase 自動 migrate
- 環境變數管理密鑰與邀請碼

技術架構

Browser
Applicant / Admin / Lab Staff

→

Django
Views · Forms · Templates

→

Service Layer
狀態轉換 · 權限副作用

→

PostgreSQL
Request · Schedule · Results

部署設定

Gunicorn 啟動 Django
WhiteNoise 提供 static files
Heroku Postgres 儲存持久資料

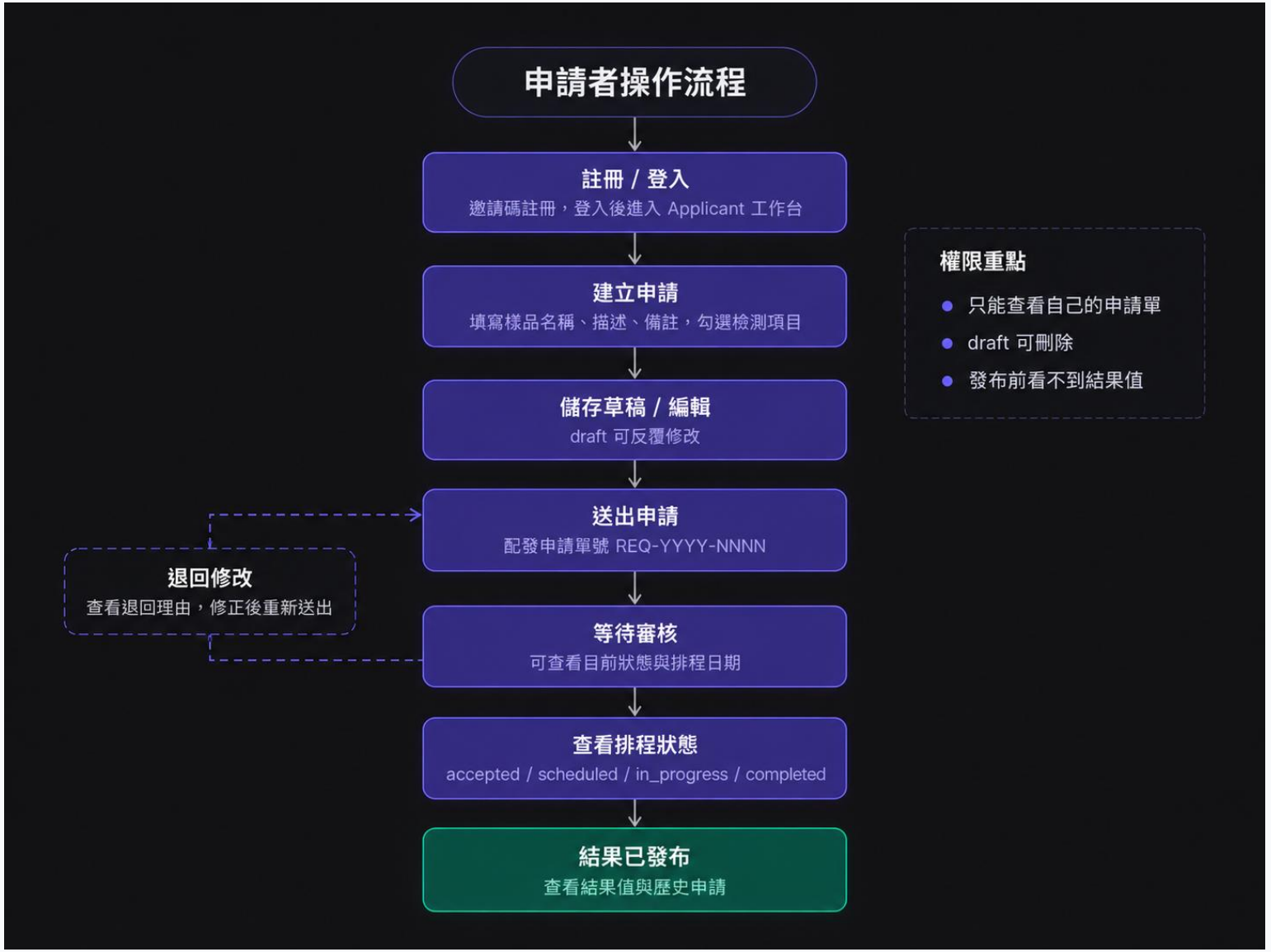
安全設定

SECRET_KEY / DATABASE_URL / 邀請碼放環境變數
HTTPS / Secure Cookie / CSRF 設定

專案結構流程圖｜整體運作



專案結構流程圖｜申請者


申請者操作流程
註冊 / 登入
邀請碼註冊，登入後進入 Applicant 工作台
建立申請
填寫樣品名稱、描述、備註，勾選檢測項目
儲存草稿 / 編輯
draft 可反覆修改
送出申請
配發申請單號 REQ-YYYY-NNNN
退回修改
查看退回理由，修正後重新送出
等待審核
可查看目前狀態與排程日期
查看排程狀態
accepted / scheduled / in_progress / completed
結果已發布
查看結果值與歷史申請
權限重點
只能查看自己的申請單
draft 可刪除
發布前看不到結果值

專案結構流程圖｜Admin



專案結構流程圖｜實驗人員



程式設計重點

身份與權限

- 登入後依群組導向對應工作台
- Applicant / Lab Staff 在 queryset 層限制可見資料
- 跨角色工作台回 403；他人資料回 404

狀態轉換

- Request.status 是流程的單一來源
- 所有合法轉換集中在 service layer
- 每次轉換都寫入 RequestStatusLog

排程與結果

- 一張 Request 對應一筆 Schedule
- Lab Staff 可暫存結果；所有結果填完才能完成
- Admin 發布後 Applicant 才可見結果

可重現的 Demo

- seed_demo 建立完整示範資料
- reset_demo 會清資料再重新 seed
- 演練失敗可以快速回復到同一個起點

推送前怎麼測試？

自動測試

- 執行：python manage.py test
- 合法 / 非法狀態轉換
- 身份與權限（403 / 404）
- 配號規則與結果完整性
- 註冊邀請碼、seed、reset

推送前流程

- python manage.py check
- python manage.py test
- python manage.py seed_demo
- 本機走一次主要劇本，再 push GitHub / Heroku

為什麼不是 pytest？

- 展示專案使用 Django 原生 TestCase 已足夠
- 減少 pytest / pytest-django 額外依賴
- 統一用 manage.py test，部署環境更單純

Demo 資料怎麼灌？怎麼 Reset？

seed_demo

- 建立 Admin / LabStaff / Applicant 三個群組
- 建立 demo_admin、lab1、lab2、applicant1、applicant2
- 建立 TestItem 與示範申請單
- 可重複執行，不會一直產生重複資料

reset_demo

- 沒有 --yes：只預覽將刪除的資料
- 加上 --yes：刪除業務資料與非 superuser 帳號
- 接著重新呼叫 seed_demo
- 上課前一小時執行一次

Heroku：heroku run -a lms-lite-demo "python manage.py reset_demo --yes"

實際程式碼｜身份與導頁

mixins.py × roles.py ×

```python
"""角色判定（簡報 §3）。

角色以 Django Group 表示；superuser 視同 Admin。
同時屬於多個群組時，依 Admin > LabStaff > Applicant 的順序選擇。
"""

ADMIN = "Admin"
LAB_STAFF = "LabStaff"
APPLICANT = "Applicant"

ROLE_PRIORITY = (ADMIN, LAB_STAFF, APPLICANT)

ROLE_LABELS = {
    ADMIN: "Admin",
    LAB_STAFF: "實驗人員",
    APPLICANT: "申請者",
}

ROLE_DASHBOARD = {
    ADMIN: "lab:admin_dashboard",
    LAB_STAFF: "lab:lab_dashboard",
    APPLICANT: "lab:applicant_dashboard",
}


def get_role(user):
    """回傳使用者的角色名稱；未登入或不屬於任何角色群組時回傳 None。"""
    if not user.is_authenticated:
        return None
    if user.is_superuser:
        return ADMIN
    # 同一個 request 內重複呼叫時不重查資料庫
    if not hasattr(user, "_role_cache"):
        names = set(user.groups.values_list("name", flat=True))
        user._role_cache = next((r for r in ROLE_PRIORITY if r in names), None)
    return user._role_cache

```

實際程式碼｜身份與導頁

mixins.py × roles.py ×

```python
"""View 層的共用權限檢查（簡報 §3）。

權限一律在這裡判斷，template 只負責顯示。
- 未登入：導向登入頁。
- 已登入但角色不符：403。
- 存取他人資料的 404 由各 view 以使用者過濾 queryset 達成。
"""
from django.contrib.auth.mixins import LoginRequiredMixin
from django.core.exceptions import PermissionDenied

from .roles import ADMIN, APPLICANT, LAB_STAFF, get_role


class RoleRequiredMixin(LoginRequiredMixin):
    required_role = None

    def dispatch(self, request, *args, **kwargs):
        if not request.user.is_authenticated:
            return self.handle_no_permission()
        if get_role(request.user) != self.required_role:
            raise PermissionDenied("您沒有進入此頁面的權限。")
        return super().dispatch(request, *args, **kwargs)


class AdminRequiredMixin(RoleRequiredMixin):
    required_role = ADMIN


class LabStaffRequiredMixin(RoleRequiredMixin):
    required_role = LAB_STAFF


class ApplicantRequiredMixin(RoleRequiredMixin):
    required_role = APPLICANT

```

實際程式碼｜身份與導頁

mixins.py × roles.py × settings.py × services.py × views.py ×

```python
from django.contrib import messages
from django.db import transaction
from django.db.models import Count, F
from django.shortcuts import get_object_or_404, redirect
from django.utils import timezone
from django.views import View
from django.views.generic import FormView, TemplateView

from accounts.mixins import AdminRequiredMixin, ApplicantRequiredMixin, LabStaffRequiredMixin

from . import services
from .forms import (
    AdminRequestFilterForm,
    ApplicantHistoryFilterForm,
    RequestForm,
    ResultForm,
    ReturnForm,
    ScheduleForm,
)
from .models import Request, Schedule
from .services import TransitionError

S = Request.Status


# ---------- 申請者 ----------


class OwnRequestMixin(ApplicantRequiredMixin):
    """只取得自己的申請單；他人的申請單一律 404（簡報 §3）。"""

    def get_own_request(self):
        return get_object_or_404(Request, pk=self.kwargs["pk"], applicant=self.request.user)

```

實際程式碼｜狀態轉換 Service

mixins.py × roles.py × settings.py × services.py × views.py × models.py ×

```
113
114 class RequestStatusLog(TimeStampedModel):
115     """狀態歷程（簡報 §4.5），每次狀態轉換寫一筆。"""
116
117     request = models.ForeignKey(Request, on_delete=models.CASCADE, related_name="status_logs", verbose_name="申請單")
118     from_status = models.CharField("原狀態", max_length=20, choices=Request.Status.choices, blank=True)
119     to_status = models.CharField("新狀態", max_length=20, choices=Request.Status.choices)
120     actor = models.ForeignKey(
121         settings.AUTH_USER_MODEL, on_delete=models.PROTECT, related_name="+", verbose_name="操作者"
122     )
123     comment = models.TextField("說明", blank=True)
124
125     class Meta:
126         ordering = ["created_at", "id"]
127         verbose_name = "狀態歷程"
128         verbose_name_plural = "狀態歷程"
129
130     def __str__(self):
131         return f"{self.from_status or '—'} → {self.to_status}"
```

實際程式碼｜狀態轉換 Service

mixins.py × roles.py × settings.py × services.py × views.py × models.py ×

```
70
71  def _transition(req, action, actor, comment="", side_effect=None):
72      from_states, to_status, who = TRANSITIONS[action]
73      with transaction.atomic():
74          locked = _lock(req)
75          if not _actor_allowed(locked, actor, who):
76              raise TransitionError("您沒有執行此操作的權限。")
77          if locked.status not in from_states:
78              raise TransitionError(
79                  f"申請單目前為「{locked.get_status_display()}」，無法{ACTION_LABELS[action]}。"
80              )
81          from_status = locked.status
82          if side_effect:
83              side_effect(locked)
84          locked.status = to_status
85          locked.save()
86          _log(locked, from_status, to_status, actor, comment)
87      req.refresh_from_db()
88      return req
```

實際程式碼｜測試、Seed 與 Reset

mixins.py × roles.py × settings.py × services.py × views.py × models.py × seed_demo.py × reset_demo.py × test_admin.py × test_lab.py ×

```
115  class LabViewTests(LabFixture):
116      def url(self, name, sched=None):
120
121      def setUp(self):
123
124      def post_results(self, req, action, values):
130
131      def test_playbook_8_6(self):
168
169      def test_dashboard_sections(self):
185
186      def test_other_staff_schedule_404(self):
187          req = self.started_request()
188          sched = req.schedule
189          self.client.force_login(self.other_staff)
190          self.assertEqual(self.client.get(self.url("detail", sched)).status_code, 404)
191          self.assertEqual(self.client.post(self.url("start", sched)).status_code, 404)
192          item = req.items.first()
193          resp = self.client.post(self.url("results", sched), {"action": "save", f"value_{item.pk}": "X"})
194          self.assertEqual(resp.status_code, 404)
195          item.refresh_from_db()
196          self.assertEqual(item.result_value, "")
197
198      def test_other_roles_403(self):
199          sched = self.scheduled_request().schedule
200          for user in (self.alice, self.admin):
201              self.client.force_login(user)
202              for url in (self.url("dashboard"), self.url("detail", sched)):
203                  self.assertEqual(self.client.get(url).status_code, 403)
204              self.assertEqual(self.client.post(self.url("start", sched)).status_code, 403)
205
206      def test_actions_are_post_only(self):
207          sched = self.started_request().schedule
208          for name in ("start", "results"):
209              self.assertEqual(self.client.get(self.url(name, sched)).status_code, 405)
```

實際程式碼｜測試、Seed 與 Reset

mixins.py | roles.py | settings.py | services.py | views.py | models.py | seed_demo.py | reset_demo.py | test_admin.py | test_results.py

```python
"""切片 5：Admin 發布、申請者結果頁、歷史查詢，以及完整的 §8-9 權限檢查。"""
import os
from datetime import datetime
from unittest import mock

from django.contrib.auth.models import User
from django.urls import reverse

from . import services
from .models import Request
from .test_lab import LabFixture
from .test_requests import TPE
from .tests import seed

S = Request.Status


def set_submitted_at(req, *args):


class PublishServiceTests(LabFixture):
    def test_publish(self):

    def test_publish_wrong_state(self):
        for req in (
            self.submitted_request(name="已送出"),
            self.scheduled_request(name="已排程"),
            self.started_request(name="實驗中"),
            services.publish(self.completed_request(name="已發布"), self.admin),
        ):
            self.assert_rejected(req, services.publish, self.admin)
        self.assertEqual(Request.objects.filter(published_at__isnull=False).count(), 1)
```

實際程式碼｜測試、Seed 與 Reset

mixins.py × roles.py × settings.py × services.py × views.py × models.py × seed_demo.py × reset_demo.py ×

```
18  # Admin 群組在 /admin/ 只能管理這三個 model（app_label, model）
19  ADMIN_MODELS = [("auth", "user"), ("auth", "group"), ("lab", "testitem")]
20  ADMIN_ACTIONS = ["view", "add", "change", "delete"]
21
22  USERS = [
30
31  TEST_ITEMS = [
86
87  RETURN_REASON = "樣品描述不足，請補充保存條件與樣品數量後重新送出。"
88
89  DEMO_REQUESTS = [
119
120 # 排程：預設指派 lab1、日期為執行 seed 當天（實驗人員「今日」區有內容）；例外如下
121 DEFAULT_STAFF = "lab1"
122 SCHEDULE_OVERRIDES = {
123     # 樣品名稱: (實驗人員, 距今天數)
124     "紅茶包": ("lab1", 1),
125     "白米": ("lab2", 0),
126 }
127
128 # 示範結果值：樣品名稱 → {項目: (結果值, 備註)}；一律為 "ND" 或不低於該項定量極限的純數字
129 DEMO_RESULTS = {
130     "雞蛋": {"鉛（Pb）": ("ND", "低於定量極限")},
131     "鮮乳": {
132         "鉛（Pb）": ("ND", "低於定量極限"),
133         "水分": ("87.6", ""),
134         "pH 值": ("6.7", "25°C"),
135         "三聚氰胺": ("ND", "低於定量極限"),
136     },
137     "綠茶粉": {
138         "鉛（Pb）": ("0.32", ""),
139         "鎘（Cd）": ("ND", "低於定量極限"),
140         "水分": ("4.8", ""),
141     },
142 }
```

實際程式碼｜測試、Seed 與 Reset

mixins.py × roles.py × settings.py × services.py × views.py × models.py × seed_demo.py × reset_demo.py ×

```python

from django.contrib.auth.models import User
from django.core.management import call_command
from django.core.management.base import BaseCommand, CommandError
from django.db import transaction

from lab.models import Request, TestItem


class Command(BaseCommand):
    help = "重置展示資料：刪除全部業務資料與非 superuser 帳號，再執行 seed_demo（須加 --yes）"

    def add_arguments(self, parser):
        parser.add_argument("--yes", action="store_true", help="確認執行重置")

    def handle(self, *args, **options):
        requests = Request.objects.all()
        test_items = TestItem.objects.all()
        users = User.objects.filter(is_superuser=False)
        summary = f"申請單 {requests.count()} 張、檢測項目 {test_items.count()} 筆、非 superuser 帳號 {users.count()} 個"

        if not options["yes"]:
            raise CommandError(f"將刪除：{summary}。確認無誤請加 --yes 重新執行；未刪除任何資料。")
        if not os.environ.get("DEMO_SEED_PASSWORD", ""):
            raise CommandError("環境變數 DEMO_SEED_PASSWORD 未設定，未刪除任何資料。")

        with transaction.atomic():
            # 先刪申請單：CASCADE 帶走項目、排程、歷程，解除它們對 User、TestItem 的 PROTECT
            requests.delete()
            test_items.delete()
            users.delete()
            self.stdout.write(f"已刪除：{summary}")
            call_command("seed_demo", stdout=self.stdout, stderr=self.stderr)

        self.stdout.write(self.style.SUCCESS("reset_demo 完成。"))
```

開發過程中遇到的難點｜角色與權限

問題

- Applicant、Lab Staff、Admin 看的是同一套資料，但權限不同
- 展示時志願者可能暫時同時屬於 Applicant 與 LabStaff
- 如果只在前端隱藏按鈕，直接輸入 URL 仍可能看到別人的資料

解法

- 角色判定集中處理，定義群組優先序
- 工作台權限用 mixin / decorator
- 資料存取再用 queryset 綁 user / assigned_staff
- 跨角色回 403；他人資料回 404，避免洩漏資料存在

開發過程中遇到的難點｜狀態一致性與展示資料

狀態一致性

- Request 有完整狀態機，Schedule 不再另外存一套 status
- 開始 / 完成只寫 started_at、completed_at
- 所有狀態轉換集中到 service layer，避免不同 View 各自改狀態

展示資料可重現

- Heroku 上的資料會被實際參與者操作
- 展示前需要固定帳號、固定主檔、固定各狀態申請單
- reset_demo 清除資料後自動 seed，確保每次排練都回到同一個起點

總結

功能

- 申請、退回、受理、排程、實驗、發布、查詢
- 三種身份工作台
- 歷史申請與狀態時間軸

程式

- Django auth + Groups
- Service layer 狀態機
- RequestStatusLog
- PostgreSQL / Heroku

品質與展示

- Django TestCase
- seed_demo / reset_demo
- 權限 403 / 404
- 可重複演練

Q&A

歡迎提問

Thank You

從申請到結果，讓每一步都能被看見。

2026/10/17